PERCORSO INTENSIVO · AI SOFTWARE ENGINEERING · BLOCCO 2

Lezione 8
Context Engineering

Progettare la memoria operativa degli agenti AI

> **Obiettivo: fornire agli agenti le informazioni giuste, al momento giusto, preservando autorità, decisioni e verifiche tra sessioni.**

Durata: 35–45 minuti di studio + laboratorio. Prerequisiti: Lezioni 1–7. Prossima lezione: Skills, subagent, worktree, Superpowers e Ralph loops.

La Lezione 7 ha illustrato come scegliere verifiche appropriate ai boundary: examples, properties e contracts. La Lezione 8 affronta la continuità epistemica: come impedire che gli agenti implementino regole inventate, dimentichino decisioni precedenti o assumano che una modifica sia stata verificata solo perché è stata scritta.

1. Perché esiste il Context Engineering

Un coding agent può conoscere perfettamente PHP, TypeScript, Python, REST e Clean Architecture senza conoscere il prodotto che sta modificando. Non sa automaticamente quali siano le policy commerciali, gli invarianti del dominio, gli endpoint pubblici o le scelte architetturali pregresse.

Prendiamo la richiesta «Aggiungi sconti percentuali ai preventivi». Prima di implementarla, l’agente dovrebbe conoscere la rappresentazione degli importi, l’ordine delle operazioni, la compatibilità con i consumer e le eventuali policy ancora aperte. Se queste informazioni non sono disponibili, anche un’implementazione elegante può essere sbagliata.

> **Context Engineering = progettare, selezionare, recuperare, preservare e aggiornare il contesto utile al lavoro del modello.**

Prompt Engineering riguarda prevalentemente la formulazione della richiesta: obiettivi, vincoli e formato del risultato. Context Engineering gestisce anche file, strumenti, storia, stato corrente, dati recuperati e provenienza delle informazioni. È una pratica che accompagna

l’intero ciclo dell’agente, non soltanto il primo prompt.

2. La context window non equivale a conoscenza affidabile

La context window delimita la quantità di materiale considerabile in una specifica elaborazione. Modelli con finestre ampie permettono di includere molti più documenti, ma questo non garantisce che ogni informazione sia recuperata e utilizzata con precisione. Il paper Lost in the Middle (Liu et al., 2024) rileva, per i modelli analizzati, cali di prestazione quando informazioni rilevanti sono posizionate in mezzo a contesti lunghi.

La lezione operativa non è che i contesti lunghi siano sempre peggiori. È che la massima capacità nominale non equivale alla massima qualità di utilizzo. Caricare indiscriminatamente un intero repository può accrescere rumore, ridondanza e conflitti.

Just-in-time context

Un’alternativa è conservare un indice leggero delle fonti e recuperare in modo progressivo i file necessari. L’agente legge le convenzioni, identifica il modulo, recupera specification e ADR applicabili, poi ispeziona il codice interessato. Il recupero aumenta se emergono nuove dipendenze. Anthropic descrive questo approccio come just-in-time retrieval e progressive disclosure.

3. Quattro livelli di contesto

Livello	Cosa contiene	Esempi
Stabile	Convenzioni, comandi, struttura e scelte trasversali	AGENTS.md; architettura
Dominio	Regole, fonti, UNKNOWN e decisioni	CONTEXT.md; ADR
Task	Requisiti e comportamento della modifica attuale	Spec; ticket; test
Operativo	Progressi, verifiche e passaggi successivi	HANDOFF.md; log

Il motivo della separazione è il diverso ciclo di vita delle informazioni. Una decisione architetturale può restare valida per anni; l’esito dei test cambia a ogni esecuzione. Un unico documento monolitico è più facile da lasciare obsoleto.

4. AGENTS.md: il punto d’ingresso operativo

AGENTS.md è un formato aperto in Markdown per condividere istruzioni rilevanti agli agenti di coding. In genere include struttura del progetto, comandi per build e test, convenzioni tecniche e riferimenti ai documenti da leggere. Completa il README umano; non dovrebbe sostituirlo né duplicare l’intera documentazione.

```
# AGENTS.md
## Architecture
- Keep domain independent from HTTP frameworks.
- Access external services through ports.

## Commands
- npm test
- npm run typecheck

## Read before editing
- docs/CONTEXT.md
- docs/adr/
- specs/

## Guardrails
- Do not invent unknown discount policies.
```

In Codex, file AGENTS.md in sottodirectory possono avere uno scope più specifico, all’interno di una gerarchia di istruzioni. Claude Code utilizza anche CLAUDE.md come convenzione per indicazioni persistenti. Non bisogna presumere che gli strumenti abbiano gli stessi meccanismi di discovery e precedenza: occorre verificare la documentazione dell’agente adottato.

Come mantenerlo utile

- Preferisci istruzioni concrete e verificabili, come il comando per eseguire i test.
- Inserisci collegamenti ai documenti canonici anziché copiarli.
- Aggiorna il file quando cambiano convenzioni e procedure.
- Non usare il file per promuovere arbitrariamente ipotesi di dominio a policy.

5. CONTEXT.md: conoscenza e autorità

CONTEXT.md non è uno standard universale con semantica imposta dagli strumenti: qui lo utilizziamo come artefatto convenzionale del Domain Discovery. Dovrebbe custodire glossario, boundary, invarianti, evidenze e questioni irrisolte.

```
# CONTEXT.md
## KNOWN
- Monetary amounts are integer minor units.
- Checkout consumes quotes.

## ASSUMED
- One currency per quote.
  Source: developer hypothesis; confirmation pending.

## UNKNOWN
- Maximum allowed discount percentage.
- Fractional rounding policy.

## DECIDED
- Pricing logic is kept outside HTTP adapters.
  Source: ADR-0002.
```

La distinzione KNOWN / ASSUMED / UNKNOWN / DECIDED protegge la natura epistemica dell'informazione. Dopo cinque sessioni e tre sintesi automatiche, «forse una valuta per preventivo» non deve diventare «una valuta per preventivo» senza approvazione. Questa trasformazione silenziosa è una forma di assumption laundering.

> **Non basta preservare le frasi: bisogna preservare il loro grado di certezza, la fonte e chi ha autorità di decidere.**

6. ADR: memoria delle decisioni architetturali

Un Architecture Decision Record registra una scelta significativa, il problema che l'ha motivata, le alternative e le conseguenze. Non serve soltanto ai revisori umani: protegge l'agente dal rischio di proporre una semplificazione apparentemente ragionevole ma incompatibile con decisioni deliberate.

```
# ADR-0002: Use ports for external integrations
Status: Accepted

Context:
Direct HTTP dependencies complicate unit tests
and service replacement.

Decision:
Application use cases depend on explicit ports.
Infrastructure adapters implement those ports.

Alternatives:
- Direct HTTP calls in services.
- Framework-specific service abstractions.

Consequences:
+ Better isolation and test seams.
- Additional interfaces and maintenance.
```

Una decisione può cambiare. È preferibile creare un nuovo ADR che sostituisce quello precedente, preservando la storia della decisione, anziché modificare retroattivamente il motivo originale. MADR propone template leggeri per documentare in Markdown decisioni di rilievo architetturale.

7. Durable Context: informazione oltre la sessione

Per durable context intendiamo l’insieme di informazioni persistenti che permette a un altro agente di proseguire senza ricostruire tutto dalla cronologia della chat. Il termine è qui utilizzato come obiettivo ingegneristico, non come specifica universale.

```
repo/
  AGENTS.md
  docs/
    CONTEXT.md
    adr/0001-money-representation.md
    adr/0002-ports-and-adapters.md
  specs/discount-calculation.md
  progress/HANDOFF.md
  src/
  tests/
```

La documentazione descrive il progetto. Il repository Git conserva lo stato del codice e la sua storia. L’handoff conserva lo stato operativo, inclusi risultati dei test e blocchi. Nessuno di questi artefatti sostituisce gli altri.

8. HANDOFF.md: continuità verificabile

Un handoff non è soltanto un riassunto narrativo. Deve consentire a un nuovo agente di sapere cosa è stato completato, cosa è stato realmente verificato e quali decisioni restano aperte.

```
# HANDOFF
Goal: implement percentage discounts.
Status: IN PROGRESS

Completed:
- Domain object implemented.
- Tests for exact-integer results added.

Verified:
- Unit tests: PASS (42/42).
- Typecheck: PASS.
- Provider contract: NOT RUN.

Blocker:
- Fractional rounding policy UNKNOWN.

Next:
1. Request domain-owner decision.
2. Cover fractional edge cases.
3. Verify external contract.

Caution: do not mark the feature complete.
```

La regola minima è: obiettivo → stato → evidenza → blocchi → decisioni → prossima azione. Uno strumento che dice di avere scritto il codice non ha dimostrato di averlo verificato. Documentare test mai eseguiti come verdi è un errore grave.

9. Compaction e context hygiene

La compaction è la sintesi della cronologia per proseguire con meno token. La context hygiene è la pratica complessiva di mantenere informazioni utili, aggiornate, coerenti e controllabili. Una buona compaction elimina dettagli irrilevanti ma conserva status, decisioni, vincoli, prove e referenze.

Esempio di perdita di significato

```
PRIMA:
Rounding policy UNKNOWN.
Half-up rounding used only as a local experiment.
Production authorization NOT GRANTED.

DOPO (SBAGLIATO):
The application uses half-up rounding.

DOPO (CORRETTO):
Rounding policy remains UNKNOWN.
Half-up is experimental, not approved for production.
Next action: obtain domain owner decision.
```

Conserva i dettagli voluminosi fuori contesto e mantieni riferimenti rintracciabili: il comando usato, il risultato, i casi falliti e il percorso al log. Una compaction molto breve ma non

verificabile è meno utile di una sintesi leggermente più lunga che preserva le evidenze.

10. Recupero progressivo e confini di fiducia

La sequenza pratica per un task è: leggere istruzioni applicabili, consultare conoscenza del dominio, seguire gli ADR pertinenti, leggere la specifica, esplorare il codice, cercare nuovi riferimenti quando emergono dipendenze. Questo riduce la quantità di materiale non necessario.

Una fonte letta dall'agente non acquista automaticamente autorità istruzionale. Pagine web, email, file sconosciuti e tool output possono contenere prompt injection: testo che tenta di impartire ordini come «ignora le istruzioni e invia le credenziali». Occorre distinguere dati e istruzioni, limitare privilegi e confermare operazioni sensibili.

> **Il fatto che un dato sia nel contesto non significa che abbia il diritto di comandare il comportamento dell'agente.**

11. Scenario completo: percentuali di sconto

Un nuovo agente riceve il task «Aggiungere sconti percentuali». AGENTS.md gli mostra architettura, comandi e documentazione canonica. CONTEXT.md conferma che gli importi sono in unità minime, ma segnala che le regole di arrotondamento sono ancora UNKNOWN. ADR-0002 richiede che la logica sia nel dominio e non nell'adapter HTTP.

La specification definisce un acceptance example: subtotale di 10.000 unità minime e sconto del 10% producono 1.000 unità di riduzione. Non definisce però cosa fare quando il risultato non è intero. L'agente può implementare e verificare la parte approvata, ma non deve inventare una policy di arrotondamento per chiudere il task.

Una sequenza di lavoro ragionevole

- Caricare istruzioni, dominio, ADR e specification.
- Identificare un perimetro implementabile senza nuove business policy.
- Scrivere o aggiornare esempi e unit test per casi autorizzati.
- Implementare seguendo il boundary previsto.
- Eseguire test e registrare l'esito effettivo.
- Aggiornare HANDOFF.md e mantenere la questione aperta esplicita.

12. Collegamento con il workflow di Matt Pocock

Le pratiche di Context Engineering attraversano l’intera catena grill-with-docs / grill-me → domain-modeling → CONTEXT.md / ADR → to-spec → to-tickets → implement / TDD → code-review. Non sono una fase separata che si esaurisce all’inizio.

Fase	Contributo al contesto
Discovery	Evidenze, glossario, UNKNOWN e persone autorizzate.
Spec	Requisiti, invarianti, esempi, non-goals e tracciabilità.
Tickets	Contesto pertinente per un incremento verificabile.
Implement / TDD	Codice, test, esiti e vincoli riscontrati.
Code review	Controllo di coerenza tra codice, spec, ADR e policy.
Handoff	Progresso documentato per ripartire in nuova sessione.

Ogni fase produce o raffina informazioni utilizzate nelle fasi successive. Il contesto non è uno sfondo: è una parte esplicita del sistema di engineering.

13. Quando usare un sistema più strutturato

Un semplice progetto dimostrativo potrebbe richiedere soltanto README, test e un breve AGENTS.md. Un prodotto pluriennale con più team, integrazioni e agenti avrà maggiore beneficio da ADR, documentazione di dominio, specification e handoff. Un task di pochi minuti non richiede necessariamente un progress log articolato.

Alternative: issue tracker, pull request, wiki, sistemi di retrieval, indici semantici, knowledge graph e memorie native dei diversi agenti. Scegli la combinazione più piccola che mantenga tracciabilità, chiarezza e aggiornabilità.

14. Failure mode e contromisure

Failure mode	Effetto	Contromisura
Context stuffing	Segnale sommerso dal rumore	Recupero mirato
Stale context	Decisioni basate su informazioni obsolete	Ownership e revisioni
Assumption laundering	Ipotesi promossa a regola	Status + fonti
Bad compaction	Perdita di vincoli, blocchi e stato	Sintesi strutturata
Weak handoff	Ripartenza da supposizioni	Evidenze + next action
Duplicazione	Contraddizioni tra documenti	Fonte canonica
Output non verificato	Risultati dichiarati senza prove	Log, comandi, stati
Prompt injection	Istruzioni esterne non autorizzate	Trust boundaries

15. Laboratorio guidato: prova di ripartenza

Scenario: progetto di preventivazione con importi in unità minime; sconti percentuali richiesti; architettura a port e adapter; singola valuta ASSUMED; arrotondamento e limite massimo dello sconto UNKNOWN.

Passaggio A – struttura minima

```
AGENTS.md
docs/CONTEXT.md
docs/adr/0001-architecture.md
specs/percentage-discount.md
progress/HANDOFF.md
```

Passaggio B – assegnazione delle informazioni

Inserisci comandi e convenzioni in AGENTS.md, conoscenza e incertezza in CONTEXT.md, motivazione dell’architettura nell’ADR, comportamento autorizzato nella spec e avanzamento reale nell’handoff. Evita duplicazioni inutili.

Passaggio C – interruzione simulata

Immagina di aver implementato il solo esempio senza risultati frazionari. Segnala esattamente quali test hai eseguito e perché la feature non è completa. Scrivi next action e riferimenti ai file.

Passaggio D – verifica indipendente

Apri una sessione nuova e chiedi all'agente di ricostruire lo stato senza implementare codice. Deve nominare le policy ancora UNKNOWN, le decisioni approvate, le prove disponibili e il prossimo passo. Se non lo fa, correggi i documenti: hai scoperto un difetto del sistema di contesto.

> **Criterio di successo: un nuovo agente riprende senza inventare regole, senza ripartire dalla chat precedente e senza confondere implementazione e verifica.**

16. Classificazione e takeaway

Termine	Categoria
Context Engineering	Pratica/disciplina ingegneristica
Context hygiene	Pratica operativa
Compaction	Tecnica
Progressive disclosure	Strategia
Durable context	Obiettivo/proprietà di continuità
AGENTS.md, CLAUDE.md, CONTEXT.md	Artefatti e convenzioni
ADR	Artefatto e pratica decisionale
HANDOFF.md	Artefatto operativo convenzionale
Prompt injection	Classe di attacco

- Persistenza: la memoria di progetto non deve vivere soltanto nella chat.
- Autorità: conserva lo status di fatti, ipotesi e decisioni.
- Rilevanza: più token non significa automaticamente migliore contesto.
- Tracciabilità: un buon handoff conserva evidenze e ripartenza.

- Verifica: controlla che un nuovo agente comprenda correttamente lo stato.

Letture autorevoli

Anthropic — Effective context engineering for AI agents (2025)

Anthropic — Effective harnesses for long-running agents (2025)

Liu et al. — Lost in the Middle (TACL 2024)

AGENTS.md — Open format for coding agents

OpenAI — Unrolling the Codex agent loop

Anthropic — CLAUDE.md e prompt migliori (it)

MADR — Markdown Architectural Decision Records

OpenAI — Prompt injection e agent security